

Translator vs. Challenger: Adversarial Agentic Learning for C-to-Rust Translation

Chaofan Wang, Xiaodong Gu, Yuling Shi, Chao Hu, Beijun Shen*

Shanghai Jiao Tong University

{chaofwang, xiaodong.gu, yuling.shi, ythere, bjshen}@sjtu.edu.cn

Abstract—C-to-Rust translation remains challenging due to the substantial semantic gap between the two languages. Recent experience-enhanced LLM translators improve translation quality by learning reusable insights from prior failures and repairs. Yet learned insights do not automatically constitute reusable translation knowledge: because they are derived from sparse and program-specific translation traces, they often contain missing conditions, narrow applicability boundaries, or overlooked corner cases. This limits their robustness and generalizability when applied to new translation scenarios. We present TRAIL, an adversarial agentic learning framework for robust C-to-Rust translation. TRAIL employs two collaborating agents: a Translator that derives candidate insights from translation failures and accepted repairs, and a Challenger that actively searches for weaknesses, gaps, and boundary cases through adversarial challenges. To improve the robustness of individual insights and the completeness of insight collections, TRAIL performs adversarial learning at two levels. Individual-insight adversarial learning repeatedly stress-tests each insight to refine its applicability conditions and constraints, while compositional insight adversarial learning strengthens groups of related insights by exposing conflicts, gaps, and uncovered corner cases. By repeatedly challenging learned insights and their compositions with executable counterexamples, TRAIL transforms trace-specific translation experience into robust, reusable, and generalizable translation knowledge. We evaluate TRAIL on two project-level benchmarks, CRUST-Bench and SmartC2Rust-Bench. Compared with the strongest LLM-based baseline, TRAIL achieves average relative improvements of 23.1% in syntax accuracy and 15.9% in semantic accuracy. Furthermore, the adversarially refined insights transfer effectively across benchmarks, demonstrating strong generalizability across diverse C-to-Rust translation tasks.

Index Terms—C-to-Rust Translation, Adversarial Agentic Learning, Translation Knowledge Refinement, Experience-Enhanced Agents.

I. INTRODUCTION

The growing adoption of Rust has created increasing demand for automated migration [1], [2] of existing C codebases [3], [4]. However, C-to-Rust translation remains challenging due to the substantial semantic gap between the two languages. While C relies heavily on raw pointers, manual memory management, and implicit type conversions, Rust enforces strict ownership, borrowing, and compile-time safety guarantees. Bridging this gap requires more than syntax-level translation: translators must infer and reconstruct safety-related semantics [5]–[8] that are often implicit in C programs. Consequently, rule-based techniques and zero-shot Large Language Model (LLM) generation frequently produce

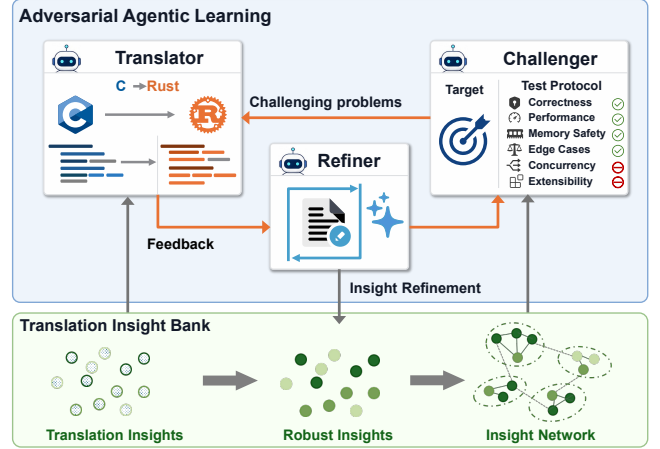


Fig. 1. Illustration of adversarial agentic learning for robust and generalizable translation insights.

uncompilable or semantically incorrect Rust code, particularly for complex, project-specific scenarios [4], [9]–[11]. These challenges motivate translation systems that can continuously acquire and refine reusable knowledge from prior translation attempts, enabling them to handle diverse coding patterns and project-specific corner cases.

Existing approaches to C-to-Rust translation have increasingly shifted from manually engineered knowledge toward automatically learned translation experience. Early systems augment LLMs with handcrafted semantic mappings [10] and translation guidelines [11] to bridge the semantic gap between C and Rust. While effective for recurring patterns, such manually authored knowledge is costly to construct, difficult to maintain, and inherently limited in its coverage of real-world C codebases. Motivated by recent advances in experience-enhanced agents [12]–[15], newer C-to-Rust translators automatically learn reusable insights from translation failures and successful repairs [9], [11], [16]. However, existing approaches primarily focus on accumulating learned insights, implicitly assuming that insights derived from past translation traces are reusable. In practice, because such insights originate from sparse and program-specific experiences, they often capture local heuristics rather than general translation knowledge, leaving missing conditions, narrow applicability boundaries, and overlooked corner cases. Consequently, insights that appear effective for previously observed failures may not generalize to

*Beijun Shen is the corresponding author.

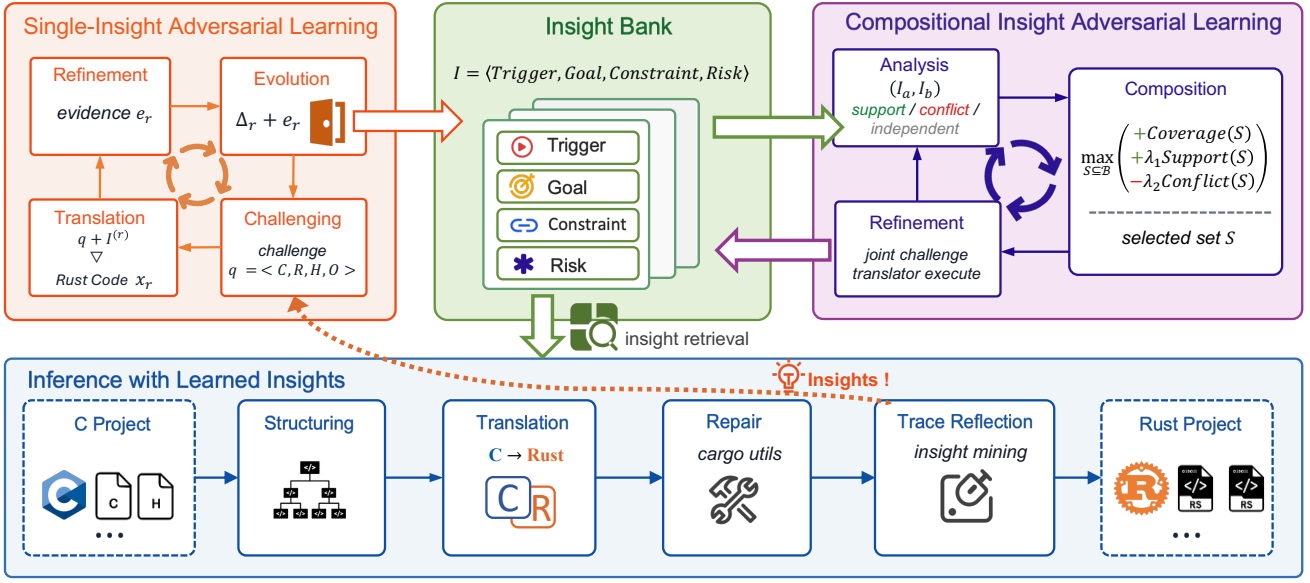


Fig. 2. Overview of TRAIL. A Challenger, Translator, and Refiner form an adversarial challenge–translate–refine loop that transforms translation experience into reusable knowledge. Adversarial refinement at both the individual-insight and compositional levels improves the robustness and completeness of learned insights for project-level C-to-Rust translation.

unseen coding patterns, alternative APIs, or interactions with other translation constraints.

To address this challenge, we propose TRAIL, an adversarial agentic learning framework that continuously refines translation insights through interaction among a Translator, a Challenger, and a Refiner. Unlike prior approaches that treat learned insights as static knowledge to be validated and stored, TRAIL formulates insight learning as a continual adversarial process: the Translator extracts candidate insights from failures and successful repairs, while the Challenger actively constructs executable counterexamples to expose missing conditions, boundary cases, and incorrect assumptions. A Refiner then converts the resulting execution feedback into structured insight updates through iterative challenge–translate–refine cycles, progressively turning trace-specific experiences into robust, reusable knowledge. Our adversarial learning operates at two levels: at the individual level, it refines each insight to produce robust translation knowledge under diverse scenarios; at the compositional level, it jointly challenges groups of related insights to surface conflicts, gaps, and interaction-induced corner cases, organizing them into a coherent insight network that improves both coverage and consistency. Figure 1 illustrates the overall idea of our framework.

We evaluate TRAIL on 100 C-to-Rust translation projects using three backend LLMs. Results show that TRAIL consistently outperforms LLM-based baselines, achieving average relative improvements of 23.1% in syntax accuracy and 15.9% in semantic accuracy. Furthermore, a transfer study on 20 projects from an independent benchmark shows that adversarially refined insights remain effective on unseen data, suggesting that TRAIL learns generalizable and reusable translation knowledge.

The main contributions of this work can be summarized as:

- We propose TRAIL, the first adversarial agentic learning framework for C-to-Rust translation. Through adversarial interaction between a Translator and a Challenger, TRAIL continuously refines learned insights and transforms trace-specific translation experience into reusable translation knowledge.
- We design a two-level adversarial learning mechanism that systematically refines learned translation insights. By challenging insights both individually and compositionally, the mechanism improves their robustness, completeness, and reusability before deployment in future translations.
- We evaluate TRAIL on 120 C-to-Rust translation projects with multiple LLM backends. Results show that TRAIL consistently outperforms state-of-the-art baselines and that adversarially refined insights transfer effectively across benchmarks, demonstrating strong generalizability.

II. TRAIL FRAMEWORK

A. Framework Overview

The core premise of TRAIL is that C-to-Rust translation experience does not directly yield reliable reusable knowledge. Although failure-and-repair traces may expose useful heuristics, the induced insights are often partial, over-specialized to specific programs, or invalid under unseen C idioms. Instead of passively accumulating such insights in a static repository, TRAIL formulates translation as an online adversarial learning process, in which insights are continuously mined from translation traces, challenged through executable tests, and iteratively refined during project migration.

TABLE I
AN EXAMPLE OF A TRANSLATION INSIGHT

Translation Insight: Pointer–Buffer Coupling
[Trigger] C code passes raw pointers, arrays, allocated buffers, string lengths, capacities, or null terminators. [Goal] Translate the pointer–buffer relation into Rust slices, <code>Vec</code>, or <code>String</code> while preserving length, capacity, and terminator semantics. [Constraint] Bounds-check every index derived from C pointer arithmetic and preserve whether a terminator is included in the logical length. [Risk] Rust code may still compile while dropping trailing-NUL handling, overrunning slices, or allocating the wrong capacity. [Tags] <code>pointer-buffer</code>, <code>ownership-transfer</code>, <code>nul-terminated-string</code>.

Figure 2 illustrates the overall architecture of TRAIL. The framework is built around structured translation insights as first-class, evolving knowledge units (Section II-B), and organized into three collaborating agents: a *Challenger*, a *Translator*, and a *Refiner*. The Challenger synthesizes executable C-to-Rust challenges that expose missing conditions, boundary cases, or implicit assumptions in existing insights. Conditioned on selected insights, the Translator generates Rust implementations that satisfy the required interface while preserving the observable behavior of the original C program. Execution results, including compilation outcomes and test executions, provide concrete feedback, which the Refiner uses to revise the corresponding insights. Together, these agents form a closed-loop challenge–translate–refine process for continuously improving translation knowledge.

To enhance robustness and completeness, TRAIL performs adversarial learning at two levels. At the *individual-insight level*, executable challenges probe whether a single insight captures a specific C-to-Rust constraint, such as pointer–buffer coupling or ownership transfer semantics (Section II-C). At the *compositional level*, multiple related insights are jointly evaluated under coordinated challenges to expose inconsistencies, coverage gaps, and interaction-induced corner cases (Section II-D). The resulting insight bank evolves into a structured, reusable knowledge base supporting project-level C-to-Rust translation and automated repair (Section II-E).

B. Translation Insight Representation

To enable systematic adversarial refinement, TRAIL represents each *translation insight* as a structured, executable knowledge unit that captures recurring C-to-Rust translation patterns extracted from translation traces.

Formally, an insight is defined as a 4-tuple:

$$I = \langle \text{Trigger}, \text{Goal}, \text{Constraint}, \text{Risk} \rangle \quad (1)$$

where *Trigger* specifies the applicability conditions under which the insight is activated, *Goal* characterizes the intended translation effect, *Constraint* encodes semantic or structural requirements that must be preserved, and *Risk* describes potential failure modes or unintended side effects induced by applying

Algorithm 1: Adversarial Challenge–Translate–Refine loop

Input: Initial refinement target $Z^{(0)}$, maximum rounds K
Output: Refined target Z^*

```

1 for  $r \leftarrow 0$  to  $K - 1$  do
2    $q_r \leftarrow \text{CHALLENGE}(Z^{(r)});$  // Challenger probes
   a boundary
3   if  $\neg \text{SANITYCHECK}(q_r, Z^{(r)})$  then
4      $Z^{(r+1)} \leftarrow Z^{(r)};$ 
5     continue;
6    $x_r \leftarrow \text{TRANSLATE}(q_r, Z^{(r)});$  // Translator uses
    $Z^{(r)}$  as constraint(s)
7    $e_r \leftarrow \text{EXECUTE}(x_r, q_r);$  // Compile and test
   the result
8    $\Delta_r \leftarrow \text{REFINE}(Z^{(r)}, q_r, x_r, e_r);$  // Refiner
   attributes failures
9    $Z^{(r+1)} \leftarrow \text{EVIDENCEGATE}(Z^{(r)}, \Delta_r, e_r);$  // Accept
   supported updates only
10  $Z^* \leftarrow Z^{(K)};$ 
11 return  $Z^*;$ 

```

the insight. In addition, each insight is annotated with a set of lightweight semantic tags to facilitate retrieval and cross-insight composition. Table I presents a representative buffer-related insight.

C. Single-Insight Adversarial Learning

TRAIL treats each learned translation insight as a falsifiable hypothesis and subjects it to iterative adversarial testing. The objective of single-insight adversarial learning is to actively construct counterexamples that expose the boundary conditions and failure modes of an insight, thereby improving its robustness and generality.

Given an insight I , TRAIL performs up to K rounds of counterexample-driven refinement. In each round, the Challenger generates an executable C-to-Rust challenge targeting the applicability boundary of I , the Translator solves it under I and the required Rust interface, and the Refiner attributes compiler or test evidence to insight deficiencies. An evidence gate then determines whether the insight is updated or retained unchanged, and the resulting version is carried to the next round. After K rounds, the final refined version is admitted to the insight bank, while candidates lacking sufficient tool-observed evidence are discarded. Algorithm 1 summarizes this Challenge–Translate–Refine loop.

Specifically, each adversarial refinement round proceeds through four steps:

1) **Challenge Generation:** The goal of challenge generation is to construct test cases that maximize the likelihood of falsifying an incomplete insight. Given the current insight instance $I^{(r)} = \langle T_I, G_I, \Phi_I, \rho_I \rangle$, the Challenger first derives an activation predicate $a_I = \text{PREDICATE}(T_I)$ that identifies programs capable of triggering the targeted transformation behavior, and extracts adversarial boundary conditions $b_I = \text{BOUNDARY}(\Phi_I, \rho_I)$ that characterize critical edge cases implied by the insight specification.

Based on a_I and b_I , the Challenger instantiates a C-to-Rust challenge $q = \langle C_q, R_q, H_q, O_q \rangle$, where C_q specifies the C program exercising the targeted idiom, R_q fixes the Rust-facing interface constraints, H_q defines the execution harness for invoking the translated implementation, and O_q encodes the expected C semantics as executable assertions. By combining activation conditions with boundary constraints under both source-language semantics and target-interface requirements, the generated challenge is designed to expose missing preconditions, over-generalized assumptions, and unhandled corner cases in the current insight.

Before execution, TRAIL applies three sanity checks to filter generated challenges, ensuring execution budget is spent on valid and diagnostic cases. LEAK removes duplicates and near-duplicates, ACTIVATES verifies triggers and boundary conditions, and TOOLCHECK ensures executability by checking that the C snippet runs, the Rust harness type-checks against a stub for R_q , and the oracle passes under `cargo test`.

2) **Insight-Constrained Translation:** Each challenge is evaluated by the Translator with the current insight $I^{(r)}$ injected as an active constraint. Conditioned on $I^{(r)}$, it synthesizes Rust code that satisfies the required interface R_q while preserving the observable behavior of C_q .

The resulting implementation is validated via `cargo check` and `cargo test`. If both succeed, the challenge is marked as solved, and TRAIL records the implementation and passing trace as positive evidence that $I^{(r)}$ covers the targeted behavior. Otherwise, compilation or test failures are treated as counterexamples; the failed program, compiler diagnostics, interface mismatches, and assertion failures are forwarded to the Refiner to update $I^{(r)}$.

3) **Failure-Driven Refinement:** When a challenge reveals a failure, the Refiner performs failure attribution to establish a diagnostic link between observed errors and the targeted insight. Insight updates are proposed only when grounded in concrete evidence, including compiler diagnostics, failing assertions, or execution traces.

Attributed failures are categorized into four insight-level deficiencies, each mapped to a field-level update. A *Trigger gap* denotes missing applicability conditions (e.g., requiring a visible length or terminator for a pointer rule), refining the *Trigger*. A *Goal mismatch* indicates an incorrect or incomplete objective (e.g., preserving bytes but violating the Rust-facing API contract), revising the *Goal*. A *Constraint violation* captures insufficient or overly permissive constraints (e.g., unchecked slice construction or violated ownership assumptions), strengthening the *Constraint*. A *Missing risk* records unmodeled failure modes (e.g., passing `cargo check` while mishandling trailing NUL bytes, introducing unintended aliasing, or altering allocation capacity), extending the *Risk* with explicit guards or failure conditions.

Refinement is triggered only by actionable failures; translator noise, invalid challenges, and successful executions do not produce updates.

4) **Evidence-Gated Insight Evolution:** The evidence gate validates each refinement proposal via replay-based verifi-

cation. Given a proposed update Δ_r , TRAIL applies it to the current insight $I^{(r)}$ to obtain a candidate $\hat{I}^{(r+1)}$. The Translator then re-executes the same challenge under $\hat{I}^{(r+1)}$ as an active constraint.

If the replay succeeds, the update is accepted and $\hat{I}^{(r+1)}$ becomes the insight for the next round. Otherwise, the proposal is rejected and $I^{(r)}$ is retained. Insights remain unchanged when (i) the original challenge already succeeds under $I^{(r)}$, or (ii) no refinement proposal is generated by the Refiner. After the final round, the latest accepted version is taken as the refined insight.

D. Compositional Insight Adversarial Learning

Improving individual insights does not guarantee that they form a complete and coherent body of translation knowledge. In project-level C-to-Rust translation, multiple insights are often activated simultaneously along shared call chains or data flows. Although each insight may be correct in isolation, their composition can introduce conflicts, missing coordination constraints, or corner cases that are not observable in single-insight refinement. For example, an insight preserving a C terminator may conflict with another constructing a Rust slice excluding it; similarly, ownership-transfer rules may interfere with insights treating the same buffer as a borrowed view. Thus, locally correct insights may lead to inconsistent global behavior.

To address this limitation, TRAIL performs adversarial learning at the compositional level. It jointly challenges co-activated insight sets within a module and evaluates their combined constraints. Through joint conflict detection and coverage-aware set refinement, TRAIL determines safe composability, required precedence or applicability constraints, and interaction-induced translation principles.

Specifically, compositional adversarial learning proceeds through three steps:

1) **Counterfactual Interaction Analysis:** Starting from the insight bank $\mathcal{B}$, whose entries have passed single-insight adversarial learning, TRAIL analyzes pairwise interactions among insights that may co-activate in the same C-to-Rust context. For an insight pair (I_a, I_b) , TRAIL performs counterfactual reasoning [17]–[19] over their *Trigger*, *Goal*, *Constraint*, *Risk*, C-idiom tags, and Rust-facing obligations to classify their relation as *support*, *independent*, or *conflict*. The analysis asks whether removing, weakening, or reordering one insight would alter the Rust interface, ownership model, aliasing assumptions, or executable behavior required by the other, yielding support and conflict scores $s_{ab}, c_{ab} \in [0, 1]$ together with explanations.

Predicted conflicts are not immediately rejected. Instead, they are flagged for adversarial testing to distinguish genuine inconsistencies from cases that require refined applicability conditions, strengthened constraints, or explicit precedence rules. This is necessary because apparent conflicts often arise from conflating internal C representation requirements (e.g., preserving a terminator in an allocated buffer) with external

Rust API obligations (e.g., exposing only initialized data via a slice).

2) **Coverage-Aware Insight Composition:** Given the interaction graph, TRAIL constructs compact insight sets for joint adversarial testing. Each set is expected to represent a coherent C-to-Rust feature, and co-activate during module-level translation. Candidate sets are formed from related triggers and overlapping language-idiom tags, such as `pointer-buffer`, `ownership-transfer`, `nul-terminated-string`, and `api-preservation`.

To balance coverage and compatibility, TRAIL selects an insight set $S \subseteq \mathcal{B}$ by maximizing

$$\max_{S \subseteq \mathcal{B}} \underbrace{Coverage(S)}_{\text{C-to-Rust tag coverage}} + \lambda_1 \underbrace{Support(S)}_{\text{LLM support score}} - \lambda_2 \underbrace{Conflict(S)}_{\text{LLM conflict score}} \quad (2)$$

where $Coverage(S)$ measures normalized coverage of distinct C-to-Rust semantic tags, defined as $|\cup_{I \in S} tags(I)| / \min(B, |\mathcal{T}|)$. Here B denotes the maximum number of insights per joint challenge, and $\mathcal{T}$ is the tag vocabulary of the candidate set. For $|S| > 1$, $Support(S)$ and $Conflict(S)$ are defined as average pairwise LLM scores:

$$\begin{aligned} Support(S) &= \frac{2}{|S|(|S| - 1)} \sum_{a < b} s_{ab}, \\ Conflict(S) &= \frac{2}{|S|(|S| - 1)} \sum_{a < b} c_{ab}. \end{aligned} \quad (3)$$

For singleton sets, both terms are zero.

In practice, TRAIL constructs compositions via greedy optimization of Eq. 2. This approximation suffices as the objective is interpretable adversarial evaluation rather than global subset optimality. The resulting set captures realistic translation scenarios, such as helper chains that compute buffer lengths, enforce terminators, transfer ownership, and preserve stable Rust-facing APIs.

3) **Compositional Adversarial Refinement:** The selected composition S is refined via joint adversarial testing. The Challenger generates executable C-to-Rust challenges that jointly activate all insights in S and force oracle dependence on their interaction; sanity checks reject cases where insights are exercised independently. Valid challenges combine multiple interacting C-to-Rust concerns, requiring a unified Rust solution satisfying all insights.

The Translator attempts the challenge with full activation of S , and execution produces compiler and test evidence as in the single-insight setting. Success indicates safe co-activation in shared contexts. Failures are attributed by the Refiner to conflicting constraints, missing coordination conditions, incomplete precedence relations, or interaction-specific corner cases. The evidence gate admits only verdict-supported updates, including field refinements, relation updates, co-activation preconditions, or precedence rules. For instance, ownership transfer may override borrowed-view assumptions when the C caller releases the buffer, while slice construction excludes a terminator preserved internally. If a failure

exposes a non-representable pattern, the Refiner introduces a new candidate insight, which is re-entered into single-insight adversarial learning before entering the bank.

Through iterative joint challenges and evidence-driven refinement, TRAIL improves interaction coherence and the completeness of learned C-to-Rust translation knowledge.

E. Inference with Learned Insights

Building on the adversarially refined insight bank, TRAIL performs project-level C-to-Rust translation as a closed-loop inference process that interleaves insight utilization and online continual learning. The inference pipeline is implemented by three cooperating agents: the Translator, which generates initial Rust implementations; the Repairer, which fixes `cargo check` and `cargo test` failures; and the Reflector, which distills reusable translation insights from execution traces and repair trajectories.

Given a C project, TRAIL first applies macro expansion and constructs a dependency graph to derive a topological order over functions. For each function, context-relevant insights are retrieved to guide both translation and subsequent repair. The generated Rust code is first validated and iteratively corrected using compilation feedback, then integrated into the project and further improved based on test execution signals. To close the learning loop, the Reflector mines reusable insights from translation and repair traces, continuously enriching the insight bank with newly observed patterns and failure-driven refinements.

Specifically, the pipeline consists of five stages.

1) **Project Structuring for Translation:** TRAIL first constructs a translation-ready project representation and establishes a dependency-aware translation order. It macro-expands the source code and performs whole-project analysis to identify translation targets, shared declarations, Rust interfaces, and inter-function dependencies. The resulting dependency graph is topologically sorted to determine the subsequent function translation order.

2) **Insight Retrieval:** Before translating each function, TRAIL retrieves context-relevant insights from the insight bank using the target C function, Rust interface, macro-expanded surrounding context, available Rust artifacts, and prior translation or repair feedback as the query.

Retrieval proceeds in two steps. Candidate insights are first selected by matching their *Trigger* and *tags* against the query, and then re-ranked by contextual relevance, compatibility, and conflict risk estimated from compositional adversarial learning. The top- N compatible insights are retained, while high-conflict combinations are filtered out.

3) **Knowledge-Guided Translation:** For each function, TRAIL combines dependency-aware project context with retrieved insights. The context includes the target Rust interface, aligned macro-expanded C code, relevant declarations and constants, validated Rust artifacts from previously translated functions, and essential dependency information, while unrelated files are omitted or abstracted to meet prompt constraints.

Conditioned on this context, the Translator generates Rust code that conforms to the target interface and preserves the observable behavior of the original C function. Retrieved insights provide structured guidance on semantic constraints, translation patterns, and potential risks.

4) **Execution-Guided Repair:** After translation, `cargo check` is used for validation. If it succeeds, the implementation is accepted and the pipeline proceeds; otherwise, the Repairer is invoked. It performs localized fixes using compiler diagnostics, the target Rust interface, relevant C context, previous Rust attempts, and retrieved insights. This repair loop repeats until compilation succeeds or the budget is exhausted.

Once all functions compile, the integrated project is validated using `cargo test`. If tests pass, the translation is accepted; otherwise, the Repairer localizes failures using test outputs, panic traces, assertions, dependency context, and recently translated components, and performs further targeted repairs under the same constraints.

5) **Trace Reflection for Insight Mining:** TRAIL performs reflection over all execution traces. The Reflector first filters out traces that are project-specific, non-recurring, or low-signal traces. For the remaining traces, it analyzes C context, compiler diagnostics, test failures, generated candidates, and final outcomes to extract underlying conditions, objectives, constraints, or risks, which are summarized as candidate insights in the form $\langle \textit{Trigger}, \textit{Goal}, \textit{Constraint}, \textit{Risk} \rangle$.

These candidates are then screened to remove duplicates, triggerless patterns, contradictions with existing insights, or artifacts tied to project-specific structure. The remaining candidates are forwarded for adversarial learning before entering the insight bank, completing the translation–learning loop.

III. EXPERIMENTAL SETUP

We conduct experiments to evaluate the effectiveness of TRAIL, aiming to answer the following research questions:

- **RQ1 (Overall Effectiveness):** How does TRAIL perform compared with state-of-the-art baselines on project-level C-to-Rust translation?
- **RQ2 (Ablation of Key Components):** What is the contribution of insight mining and the two-level adversarial learning mechanism to translation performance?
- **RQ3 (Generalization of Learned Insights):** How well do insights learned from one set of C-to-Rust projects transfer to unseen project collections in improving translation performance?
- **RQ4 (Sensitivity Analysis):** How do key hyperparameters, including insight selection size and the number of adversarial learning rounds, affect the performance and stability of TRAIL?

A. Datasets

We evaluate TRAIL on two C-to-Rust translation benchmarks.

1) **CRUST-Bench:** CRUST-Bench [4] contains 100 real-world C projects collected from GitHub, Linux, and PostgreSQL, covering domains such as data structures, cryptography, encoding, parsing, and system utilities. Each project is paired with an idiomatic Rust interface crate and executable tests. We use CRUST-Bench as the primary benchmark for effectiveness, ablation, and sensitivity studies.

2) **SmartC2Rust-Bench:** To evaluate cross-benchmark generalization, we use the benchmark introduced by SmartC2Rust [11]. Of its 21 programs, 20 are publicly available and used in our experiments. These subjects comprise small-to-medium command-line utilities and library-style programs with build scripts, test scripts, and entry-point specifications. Unlike CRUST-Bench, which relies on Rust interface crates, SmartC2Rust-Bench evaluates translations through executable behavioral checks.

B. Models

We evaluate TRAIL with three representative LLM backends: **GPT-5.4-mini**, a compact model with strong code-generation capabilities; **Kimi-K2.5**, a large-scale model with an extended context window; and **DeepSeek-V4-Flash**, a fast inference model optimized for coding tasks. All models are accessed through OpenAI-compatible APIs. Unless otherwise specified, we use `temperature=1.0` and `max_output_tokens=128000` for all experiments.

C. Baselines

We compare TRAIL against five representative baselines spanning rule-based, hybrid, and LLM-based C-to-Rust translation systems. All baselines are evaluated using their default configurations.

- **C2Rust** [1] is a rule-based source-to-source translator that mechanically converts C code into Rust. It serves as a non-LLM baseline for transpilation-based migration.
- **C2SaferRust** [20] is a hybrid approach that first translates C into unsafe Rust using C2Rust and then employs an LLM to incrementally rewrite unsafe code into safer Rust while preserving behavior through test-based verification.
- **Direct Prompting** uses the backend LLM to translate each callable without learned insights or iterative repair, representing the model’s zero-experience translation capability.
- **Self-Repair** [4] augments direct translation with iterative compilation-driven repair (up to five rounds) but does not leverage learned insights, isolating the effect of repair-based feedback.
- **SmartC2Rust** [11] is an LLM-based translation framework that performs translation and repair using compilation and test feedback, but does not incorporate structured translation insights.

No public C-to-Rust baseline provides a reproducible structured-insight pipeline for comparison. Therefore, in RQ2 we include a variant with directly mined insights via trace reflection, serving as a non-adversarial baseline.

TABLE II
OVERALL COMPARISON OF TRAIL AND BASELINES ON 100 CRUST-BENCH PROJECTS

Model	Method	CompRate	TestRate	SafeRate	Idiomatcity	
					Lint Pass $\uparrow$	Idiom Penalty $\downarrow$
Rule-based and hybrid baselines						
—	C2Rust	98%	98%	0.13%	98	49.06
gpt-5.4-mini	C2SaferRust	98%	98%	12.28%	94	49.20
LLM-based methods						
gpt-5.4-mini	Direct Prompting	32%	24%	99.66%	34	11.98
	Self-Repair	56%	55%	99.67%	65	8.70
	SmartC2Rust	66%	40%	99.41%	67	4.77
	TRAIL (ours)	88%	69%	96.63%	88	3.95
kimi-k2.5	Direct Prompting	39%	25%	98.95%	41	13.27
	Self-Repair	76%	61%	99.03%	84	8.48
	SmartC2Rust	59%	37%	97.16%	60	5.21
	TRAIL (ours)	79%	67%	95.12%	89	4.47
deepseek-v4-flash	Direct Prompting	21%	14%	98.81%	25	11.89
	Self-Repair	56%	48%	99.08%	67	9.23
	SmartC2Rust	31%	26%	98.49%	15	7.22
	TRAIL (ours)	74%	54%	97.57%	88	4.22

* For LLM-based methods, results are grouped by backbone model, with best in **bold**.

D. Metrics

We assess translation quality from four perspectives: compilation success, behavioral correctness, safety, and idiomatcity.

1) **CompRate**: The percentage of projects that successfully pass `cargo check`, indicating successful project-level compilation.

2) **TestRate**: The percentage of projects that pass all `cargo test` cases, indicating preservation of observable program behavior.

3) **SafeRate**: The percentage of translated callables that contain no `unsafe` blocks, measuring the extent to which translations leverage Rust’s safety guarantees.

4) **Idiomatcity**: Measured using two metrics. **Lint Pass** counts the number of projects that pass `cargo clippy`, reflecting adherence to Rust coding conventions. **Idiom Penalty** follows the non-idiomatic pattern penalty defined by Rustine [21]. Higher Lint Pass and lower Idiom Penalty indicate more idiomatic Rust code.

E. Implementation Details

The insight bank is stored as structured JSON records and indexed using FAISS. Each insight’s *Trigger* and *tags* are encoded with BAAI/bge-base-en-v1.5 for similarity-based retrieval, with a default retrieval budget of $N = 5$ insights.

All agents are implemented as structured LLM calls with fixed schemas, prompts, output formats, and acceptance criteria, ensuring consistency across the full TRAIL pipeline and all ablation variants. All prompts are released in our artifact repository to support reproducibility.

For compositional adversarial learning, the backend LLM estimates support and conflict scores in Eq. 2. We set $\lambda_1 =$

$\lambda_2 = 1$ and use a conflict threshold $\tau_c = 0.7$. Candidate insights are grouped by related triggers and overlapping *tags*, and compositions are constructed greedily by iteratively adding the largest positive marginal gain until reaching $B = 3$, exhausting positive-gain candidates, or encountering only unresolved conflicts.

Unless otherwise specified, each accepted insight undergoes at most $K = 5$ adversarial refinement rounds. During execution-guided repair, compilation-level fixes are limited to five attempts per callable (following Self-Repair), and project-level test repair is capped at three attempts. A repair is accepted only if it succeeds or reduces the number of diagnostics or failing tests.

IV. RESULTS

A. RQ1: Overall Effectiveness

We evaluate TRAIL against rule-based, LLM-based, and hybrid baselines on 100 CRUST-Bench projects. Table II summarizes the results.

Across all three backbone models, TRAIL achieves the strongest LLM-based project-level correctness. With gpt-5.4-mini, it reaches 88% CompRate and 69% TestRate, surpassing SmartC2Rust’s 66% CompRate and Self-Repair’s 55% TestRate by 22 and 14 percentage points, respectively. Similar gains appear on kimi-k2.5 (79%/67% vs. 76%/61%) and deepseek-v4-flash (74%/54% vs. 56%/48%), showing that the effect is consistent across backbones.

The improvements over SmartC2Rust and Self-Repair suggest that proactive knowledge injection complements iterative repair. TRAIL mines and refines reusable insights from traces, helping prevent recurring failures rather than only fixing them after they appear.

TABLE III
ABLATION STUDY OF THE PROPOSED INSIGHT LEARNING MECHANISM, WITH CUMULATIVE REMOVAL OF EACH COMPONENT FROM TRAIL

Model	Variant	CompRate	TestRate	SafeRate	Idiomatcity	
					Lint Pass ↑	Idiom Penalty ↓
gpt-5.4-mini	TRAIL	88%	69%	96.63%	88	3.95
	w/o Compositional Insight Adversarial Learning	86%	66%	96.22%	89	4.00
	w/o Single Insight Adversarial Learning	85%	63%	96.60%	91	4.07
	w/o Insight Mining	75%	56%	93.87%	87	4.16
kimi-k2.5	TRAIL	79%	67%	95.12%	89	4.47
	w/o Compositional Insight Adversarial Learning	76%	63%	94.76%	86	4.37
	w/o Single Insight Adversarial Learning	69%	54%	94.95%	89	4.28
	w/o Insight Mining	70%	49%	94.98%	87	4.31
deepseek-v4-flash	TRAIL	74%	54%	97.57%	88	4.22
	w/o Compositional Insight Adversarial Learning	75%	51%	97.74%	90	4.26
	w/o Single Insight Adversarial Learning	70%	49%	97.73%	87	4.22
	w/o Insight Mining	68%	45%	96.85%	88	4.10

* Best results per model are in **bold**.

Rule-based and hybrid baselines show a different trade-off. C2Rust and C2SaferRust reach up to 98% CompRate and TestRate, but rely on unsafe-first transpilation and therefore suffer extremely low safety and poor idiomatcity. Thus, compile/test success alone does not characterize translation quality. Although TRAIL does not yet match their raw pass rates, it narrows the gap while avoiding their severe safety and idiomatcity degradation.

Answer to RQ1. TRAIL achieves the best overall effectiveness among LLM-based approaches across all three backend models, while preserving a better balance across functional correctness, safety, and idiomatcity than the unsafe transpilation-first baselines.

B. RQ2: Ablation of Key Components

To quantify each component’s contribution, we conduct a stepwise ablation study on CRUST-Bench. Starting from the full TRAIL framework, we progressively remove compositional insight adversarial learning, single-insight adversarial learning, and insight mining. Table III summarizes the results.

All components are beneficial, with performance degrading under each removal. Removing compositional insight adversarial learning reduces behavioral correctness across backbones, highlighting its role in coordinating related insights. In the full system, it boosts TestRate to 69%, 67%, and 54% on gpt-5.4-mini, kimi-k2.5, and deepseek-v4-flash while keeping CompRate stable. Without it, cross-insight inconsistency leads to TestRate drops and minor CompRate trade-offs (e.g., 75%→74% on deepseek-v4-flash).

Further removing single-insight adversarial learning causes larger degradation: on kimi-k2.5, TestRate/CompRate decrease from 63%/76% to 54%/69%; on deepseek-v4-flash, from 51%/75% to 49%/70%; gpt-5.4-mini shows a smaller but consistent decline (66%/86%→63%/85%), indicating that counterexample-driven refinement improves per-insight precision and robustness.

Removing insight mining yields the largest drop, with TestRate falling to 56%, 49%, and 45% on gpt-5.4-mini, kimi-k2.5, and deepseek-v4-flash. This suggests that even unrefined trace-reflected insights provide useful semantic constraints beyond a standard translation-and-repair pipeline.

Across variants, SafeRate remains stable (96.63%, 95.12%, 97.57%), and idiomatcity changes marginally, indicating functional gains do not compromise safety or code quality.

Answer to RQ2. Failure-driven mined insights already provide measurable benefits, while adversarial learning at both the single-insight and compositional levels further improves the reliability of reusable constraints, mainly by boosting TestRate without sacrificing SafeRate or idiomatcity.

C. RQ3: Generalization of Learned Insights

To evaluate cross-benchmark generalization, we transfer the CRUST-Bench insight bank to SmartC2Rust-Bench. Table IV compares *Base* without insight injection, *Direct* with target-benchmark insights, and *Transferred* with CRUST-Bench insights and no further adaptation.

Transferred improves over *Base* across models. For gpt-5.4-mini, CompRate/TestRate rises from 80%/55% to 85%/70%; for kimi-k2.5, CompRate stays at 90% while TestRate improves from 65% to 70%; for deepseek-v4-flash, CompRate rises from 75% to 80% while TestRate remains 70%. These gains indicate that the learned insights capture reusable C-to-Rust constraints beyond the source benchmark.

However, *Transferred* remains below *Direct*, which learns on SmartC2Rust-Bench itself. Compared with *Transferred*, *Direct* improves CompRate by 5 points for all models and boosts TestRate by 5, 10, and 10 points on gpt-5.4-mini, kimi-k2.5, and deepseek-v4-flash. Thus, transferred insights provide strong priors, but adversarial refinement is still needed for benchmark-specific APIs, failure modes, and boundary conditions.

TABLE IV
CROSS-BENCHMARK GENERALIZATION OF ADVERSARIALY REFINED INSIGHTS

Model	Setting	CompRate	TestRate	SafeRate	Idiomaticity	
					Lint Pass $\uparrow$	Idiom Penalty $\downarrow$
gpt-5.4-mini	Base	80%	55%	98.57%	13/20	5.17
	Direct	90%	75%	98.44%	16/20	4.72
	Transferred	85%	70%	98.62%	13/20	5.28
kimi-k2.5	Base	90%	65%	97.62%	14/20	19.13
	Direct	95%	80%	97.71%	16/20	17.40
	Transferred	90%	70%	97.55%	15/20	18.86
deepseek-v4-flash	Base	75%	70%	98.63%	16/20	2.99
	Direct	85%	80%	98.68%	17/20	2.84
	Transferred	80%	70%	98.59%	17/20	3.08

* Insights are learned on CRUST-Bench and transferred to SmartC2Rust-Bench. *Base* uses no insights, *Direct* uses target-benchmark insights, and *Transferred* uses CRUST-Bench insights.

Safety and idiomaticity remain stable under transfer: SafeRate changes marginally, Lint Pass is unchanged or slightly higher, and Idiom Penalty varies only modestly. Overall, *Transferred* improves functional correctness without degrading code quality, while *Direct* achieves the best balance.

Answer to RQ3. Transferred insights consistently match or improve the base pipeline in compilation and testing, with broadly stable safety and idiomaticity, while the full TRAIL pipeline remains strongest due to benchmark-specific adversarial refinement.

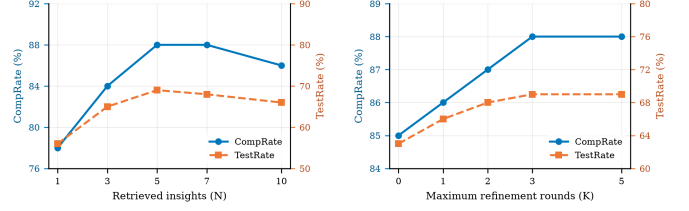
D. RQ4: Sensitivity Analysis

We study two key hyperparameters of TRAIL on CRUST-Bench with gpt-5.4-mini: the insight retrieval size (N) and the maximum number of adversarial refinement rounds (K). To isolate their effects, we vary one parameter while fixing the other ($K=5$ for sweeping N , and $N=5$ for sweeping K).

Figure 3a shows that performance benefits from a moderate retrieval budget. With $N=1$, TRAIL reaches 78% CompRate and 56% TestRate, indicating insufficient coverage of interacting C idioms. Increasing N to 3 improves results to 84%/65%, and the default $N=5$ performs best (88%/69%). Larger budgets reduce TestRate (68% at $N=7$, 66% at $N=10$) without CompRate gains, suggesting interference from irrelevant or weakly relevant constraints.

Figure 3b shows saturation in adversarial refinement. Without refinement ($K=0$), performance is 85% CompRate and 63% TestRate. One and two rounds improve results to 86%/66% and 87%/68%, while $K \geq 3$ already matches the best observed performance (88%/69%). Larger K mainly stabilizes the outcome.

Answer to RQ4. TRAIL performs best under moderate retrieval and bounded adversarial learning: $N=5$ achieves the strongest CompRate and TestRate, while most gains are realized within two to three refinement rounds, and $K=5$ serves as a conservative upper bound with stable performance.



(a) Effect of insight selection size (N). (b) Effect of adversarial rounds (K).

Fig. 3. Hyperparameter sensitivity analysis of TRAIL on CRUST-Bench with GPT-5.4-mini.

E. Qualitative Insight Evolution

We inspect two representative learning traces to understand what the adversarially refined insights actually encode.

1) *Single-Insight Adversarial Learning*: In `libvcd`, the initial pointer-buffer insight preserves buffer length and NUL termination but misses the boundary between internal storage and Rust-visible string semantics. The C code stores signals in fixed `char[N]` arrays with NUL-terminated content, while Rust conversions such as `String::from_utf8_lossy` may expose zero-padded regions. The challenge reveals that copying the full buffer is insufficient when downstream operations interpret it as a C string.

The refined insight `fixed-c-string-visible-length` strengthens the rule into a visibility-aware condition: fixed C arrays may retain zero-padded storage internally, but any Rust-visible representation, comparison, or lookup must stop at the first NUL unless the original program treats the buffer as raw bytes. The insight thus evolves from a structural constraint into an observation-boundary rule.

2) *Compositional Insight Adversarial Learning*: In `lib2bit`, the `twobitSequence` pipeline involves multiple cooperating insights: one preserves helper-call structure, and another enforces allocation size, decoding length, and NUL termination. Alone, they remain incomplete because one lacks the Rust-visible sequence length and the other does not specify when masking applies.

The compositional challenge exposes this interaction gap and aligns the insights into one policy: the Rust-visible length is `end - start`, the C buffer allocates one extra byte for the terminator, masking applies only to valid decoded bases, and the terminator is excluded from the logical sequence. The insights thus become a coordinated constraint set for the same abstraction.

V. DISCUSSION

A. Cost Analysis

We analyze TRAIL’s cost in token usage and end-to-end runtime. On CRUST-Bench with gpt-5.4-mini, the full configuration consumes 21.43M tokens (214.3K per project), compared with 19.51M tokens (195.1K per project) for the no-insight setting. The 1.92M-token increase, or 9.8% overhead, remains modest because most tokens are still spent on translation,

compiler feedback, and test-driven repair, while the compact insight bank is injected selectively.

For runtime, TRAIL takes 15,326 seconds in total (153.3 seconds per project), compared with 10,452 seconds (104.5 seconds per project) for SmartC2Rust, a $1.47\times$ slowdown mainly due to trace reflection and the adversarial challenge–translate–refine loop.

These results indicate moderate token and runtime overhead, amortized as refined insights guide later translation and repair. TRAIL is therefore most suitable for project-level migration where slightly higher latency is acceptable for improved correctness, safety, and idiomaticity, while latency-sensitive settings may rely on seeded insight banks with limited online refinement.

B. Threats to Validity

Internal Validity. TRAIL validates translations using `cargo check` and `cargo test`. While these provide objective correctness signals, they do not establish full semantic equivalence with the original C implementation. Future work will complement them with fuzzing, differential testing, and stronger API specifications.

Beyond execution-based validation, compositional insight learning depends on LLM-estimated support and conflict scores in Eq. 2, which may vary across backend models or decoding settings. We reduce this risk by using fixed schemas and prompts and by treating the scores only as a heuristic for prioritizing insight compositions before executable validation; actual insight and relation updates are accepted only when supported by compiler or test evidence.

Another threat is potential benchmark contamination during LLM pre-training. This risk is reduced because the benchmarks contain no Rust reference implementations, preventing direct memorization of target translations. However, models may still have seen the original C projects. Evaluating on private or newly released projects would further mitigate this threat.

External Validity. Our evaluation is limited to public user-level C projects. CRUST-Bench and SmartC2Rust-Bench cover diverse application domains, supporting generalization across a broad range of C-to-Rust translation tasks. However, they do not represent all migration scenarios, such as kernel or device-driver code, highly concurrent systems, projects with complex native dependencies, or large industrial codebases. Extending the evaluation to these settings remains future work.

VI. RELATED WORK

A. C-to-Rust Translation

Existing C-to-Rust translation work can be broadly grouped into rule-based and LLM-based approaches.

Rule-based methods mechanically translate C into Rust [1], [2], [22] while largely preserving the original program structure, often producing code that relies heavily on raw pointers and `unsafe` constructs. Subsequent work improves safety and idiomaticity through analyses and transformations for

ownership [7], aliasing [6], pointer safety [5], [23], API migration [8], [24]–[29], and type migration [30], [31]. However, these approaches rely on handcrafted analyses or transformation rules, limiting their generality.

LLM-based methods generate idiomatic Rust code without manually defined translation rules. Existing work improves translation via semantic guidance [32]–[34], retrieval [9], [35]–[37], execution-driven repair [11], [16], [38]–[41], and project-level context [42]–[44]. Despite these advances, ensuring reliable translation correctness remains challenging due to the substantial semantic and paradigm gap between C and Rust. A line of work leverages LLMs to enhance rule-based transpilers or static analysis, for example by refining transpiler outputs [20], injecting semantic guidance [45], [46], or performing skeleton-guided repair [10].

Empirical studies [47]–[50] and benchmarks [3], [4] further highlight persistent trade-offs among correctness, safety, and idiomaticity in C-to-Rust migration.

TRAIL belongs to the LLM-based category. Unlike prior approaches that rely on manual rules, retrieval, or repair-driven generation, TRAIL models reusable translation knowledge as executable insights and refines them through adversarial learning, improving the reliability of knowledge-guided project-level translation while achieving a better balance among correctness, safety, and idiomaticity.

B. Experience-Enhanced AI Agents

Experience enhancement equips LLM agents with reusable knowledge accumulated across prior tasks, enabling transfer beyond the current context. A line of work learns such experience from agent trajectories. For example, Reflexion [12] transforms execution feedback into verbal reflections; ExpeL [13] distills successful and failed trajectories into reusable lessons; AutoGuide [51] synthesizes context-aware guidelines; and Agent Workflow Memory [14] extracts reusable workflows from past executions.

This paradigm has also been adopted in software engineering. AgentRR abstracts interaction traces into structured experiences for reuse across similar tasks [52]. Agent KB aggregates heterogeneous trajectories into a knowledge base for cross-task retrieval [53], while SWE-Exp distills prior issue-resolution traces into actionable repair experience [15]. These systems improve reuse by storing and retrieving past trajectories or repair patterns [54], often augmented with lightweight controls such as verification [55], disagreement filtering, or retrieval-based selection.

However, such experience is typically derived from stochastic agent executions, which may encode spurious decisions, omit boundary conditions, or introduce inconsistencies across tasks, limiting reliability under distribution shifts. To address this limitation, TRAIL treats translation experience as a hypothesis to be validated: it is adversarially challenged, refined, and compositionally verified before reuse.

VII. CONCLUSION

This paper presents TRAIL, an adversarial agentic framework for C-to-Rust translation that continuously refines

reusable translation insights through Translator–Challenger interaction and evidence-based updates. By strengthening insights at both individual and compositional levels, TRAIL improves their robustness and coordination across diverse translation scenarios. Experiments on two project-level benchmarks show that TRAIL consistently outperforms existing LLM-based approaches and learns insights that generalize beyond their source programs, while maintaining stable safety and idiomaticity.

DATA AVAILABILITY

To support reproducibility, all related scripts and data are available at <https://github.com/bbzwscf/TRAIL>.

REFERENCES

- [1] “C2Rust: Migrate C code to Rust,” <https://github.com/immunant/c2rust>, 2025.
- [2] N. Shetty, N. Saldanha, and M. Thippeswamy, “Crust: Ac/c++ to rust transpiler using a “nano-parser methodology” to avoid c/c++ safety issues in legacy code,” in *Emerging Research in Computing, Information, Communication and Applications: ERCICA 2018, Volume 1*. Springer, 2019, pp. 241–250.
- [3] G. Ou, M. Liu, Y. Chen, Y. Wang, X. Peng, and Z. Zheng, “Rustreprotrans: Repository-level context code translation benchmark targeting rust,” in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2025, pp. 610–622.
- [4] A. Khatry, R. Zhang, J. Pan, Z. Wang, Q. Chen, G. Durrett, and I. Dillig, “CRUST-bench: A comprehensive benchmark for c-to-safe-rust transpilation,” in *Second Conference on Language Modeling*, 2025. [Online]. Available: <https://openreview.net/forum?id=8xofWL61S9>
- [5] M. Emre, R. Schroeder, K. Dewey, and B. Hardekopf, “Translating c to safer rust,” *Proceedings of the ACM on Programming Languages*, vol. 5, no. OOPSLA, pp. 1–29, 2021.
- [6] M. Emre, P. Boyland, A. Parekh, R. Schroeder, K. Dewey, and B. Hardekopf, “Aliasing limits on translating c to safe rust,” *Proceedings of the ACM on Programming Languages*, vol. 7, no. OOPSLA1, pp. 551–579, 2023.
- [7] H. Zhang, C. David, Y. Yu, and M. Wang, “Ownership guided C to Rust translation,” in *International Conference on Computer Aided Verification*. Springer, 2023, pp. 459–482.
- [8] J. Hong and S. Ryu, “Don’t write, but return: Replacing output parameters with algebraic data types in c-to-rust translation,” *Proceedings of the ACM on Programming Languages*, vol. 8, no. PLDI, pp. 716–740, 2024.
- [9] X. Cai, J. Liu, X. Huang, Y. Yu, H. Wu, C. Li, B. Wang, I. N. B. Yusuf, and L. Jiang, “Rustmap: Towards project-scale c-to-rust migration via program analysis and llm,” in *International Conference on Engineering of Complex Computer Systems*, 2025, pp. 283–302.
- [10] C. Wang, T. Yu, B. Shen, J. Wang, D. Chen, W. Zhang, Y. Shi, C. Xie, and X. Gu, “EvoC2Rust: A skeleton-guided framework for project-level C-to-Rust translation,” in *IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2026. [Online]. Available: <https://arxiv.org/abs/2508.04295>
- [11] M. Shiraishi, Y. Cao, and T. Shinagawa, “SmartC2Rust: Iterative, feedback-driven C-to-Rust translation via large language models for safety and equivalence,” in *Proceedings of the ACM/IEEE 48th International Conference on Software Engineering*, 2026. [Online]. Available: <https://arxiv.org/abs/2409.10506>
- [12] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflection: Language agents with verbal reinforcement learning,” *Advances in neural information processing systems*, vol. 36, pp. 8634–8652, 2023.
- [13] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, “Expel: Llm agents are experiential learners,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 19 632–19 642.
- [14] Z. Z. Wang, J. Mao, D. Fried, and G. Neubig, “Agent workflow memory,” *arXiv preprint arXiv:2409.07429*, 2024.
- [15] S. Chen, S. Lin, Y. Shi, H. Lian, X. Gu, L. Yun, D. Chen, L. Cao, J. Liu, N. Xia *et al.*, “Swe-exp: Experience-driven software issue resolution,” *arXiv preprint arXiv:2507.23361*, 2025.
- [16] S. Wang, M. Liu, G. Ou, Y. Chen, Z. Li, Y. Wang, and Z. Zheng, “Build-aware incremental c-to-rust migration via skeleton-first translation and historical knowledge reuse,” *arXiv preprint arXiv:2603.02617*, 2026.
- [17] W. Zeng, Y. Wang, C. Hu, Y. Shi, C. Wan, H. Zhang, and X. Gu, “Pruning the unsurprising: Efficient llm reasoning via first-token surprisal,” *arXiv preprint arXiv:2508.05988*, 2025.
- [18] W. Zeng, X. Zhang, Y. Shi, C. Hu, Y. Chen, B. Shen, and X. Gu, “Glimprouter: Efficient collaborative inference by glimpsing one token of thoughts,” in *Findings of the Association for Computational Linguistics, ACL 2026, San Diego, California, United States, July 2-7, 2026*, M. Liakata, V. P. Moreira, J. Zhang, and D. Jurgens, Eds. Association for Computational Linguistics, 2026, pp. 17 850–17 864. [Online]. Available: <https://doi.org/10.18653/v1/2026.findings-acl.885>
- [19] X. Zhang, W. Zeng, X. Gu, C. Hu, H. Lin, Y. Shi, M. Wang, and B. Shen, “Paratempo: Efficient parallel reasoning via temporal confidence,” *arXiv preprint arXiv:2608.16425*, 2026.
- [20] V. Nitin, R. Krishna, L. L. do Valle, and B. Ray, “C2 saferrust: Transforming c projects into safer rust with neurosymbolic techniques,” *IEEE Transactions on Software Engineering*, 2025.
- [21] S. Dehghan, T. Sun, T. Wu, Z. Li, and R. Jabbarvand, “Translating large-scale c repositories to idiomatic rust,” *arXiv preprint arXiv:2511.20617*, 2025.
- [22] X. Han, B. Hua, Y. Wang, and Z. Zhang, “Rusty: Effective c to rust conversion via unstructured control specialization,” in *2022 IEEE 22nd International Conference on Software Quality, Reliability, and Security Companion (QRS-C)*. IEEE, 2022, pp. 760–761.
- [23] M. Ling, Y. Yu, H. Wu, Y. Wang, J. R. Cordy, and A. E. Hassan, “In rust we trust: a transpiler from unsafe c to safer rust,” in *Proceedings of the ACM/IEEE 44th international conference on software engineering: companion proceedings*, 2022, pp. 354–355.
- [24] J. Hong and S. Ryu, “Concrat: An automatic C-to-Rust lock API translator for concurrent programs,” in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 716–728.
- [25] —, “To tag, or not to tag: Translating C’s unions to Rust’s tagged unions,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 40–52.
- [26] —, “Forcrat: Automatic I/O API translation from C to Rust via origin and capability analysis,” in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2025, pp. 1541–1552.
- [27] X. Wu and B. Demsky, “GenC2Rust: Towards generating generic Rust code from C,” in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 90–102.
- [28] V. Chen, A. Coughlin, and M. D. Bond, “&inator: Correct, precise c-to-rust interface translation,” *Proceedings of the ACM on Programming Languages*, vol. 10, no. PLDI, pp. 580–603, 2026.
- [29] H. Peng, B. Kasikci, G. L. Bernstein, and M. D. Ernst, “Hayroll: A modular wrapper for translating c macros and conditional compilation to rust,” *Proceedings of the ACM on Programming Languages*, vol. 10, no. PLDI, pp. 730–753, 2026.
- [30] J. Hong and S. Ryu, “Type-migrating c-to-rust translation using a large language model,” *Empirical Software Engineering*, vol. 30, no. 1, p. 3, 2025.
- [31] Q. Xu and J. Huang, “Optimizing type migration for llm-based c-to-rust translation: A data flow graph approach,” in *Proceedings of the 14th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis*, 2025, pp. 8–14.
- [32] F. Luo, K. Ji, C. Gao, S. Gao, J. Feng, K. Liu, X. Xia, and M. R. Lyu, “Integrating rules and semantics for llm-based c-to-rust translation,” in *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2025, pp. 685–696.
- [33] M. Farrukh, B. Coskun, T. Palit, and M. Polychronakis, “Safetrans: Llm-assisted transpilation from c to rust,” in *Proceedings of the 1st Workshop on Code Translation, Transformation, and Modernization*, 2026, pp. 30–37.
- [34] A. Z. Yang, Y. Takashima, B. Paulsen, J. Dodds, and D. Kroening, “Vert: Polyglot verified equivalent rust transpilation with large language models,” in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2025, pp. 1453–1463.
- [35] H. F. Eniser, H. Zhang, C. David, M. Wang, M. Christakis, B. Paulsen, J. Dodds, and D. Kroening, “Towards translating real-world code with llms: A study of translating to rust,” *arXiv preprint arXiv:2405.11514*, 2024.
- [36] Z. Yuan, W. Mao, Z. Chen, X. Shang, C. Wang, Y. Lou, and X. Peng, “Project-level c-to-rust translation via pointer knowledge graphs,” 2026. [Online]. Available: <https://arxiv.org/abs/2510.10956>

- [37] J. Feng, W. Gan, C. Gao, C. Wang, F. Luo, X. Xia, G. Li, and K. Liu, "Dependency-guided repository-level c-to-rust translation with reinforcement alignment," *arXiv preprint arXiv:2604.02852*, 2026.
- [38] M. Shetty, N. Jain, A. Godbole, S. A. Seshia, and K. Sen, "Syzygy: Dual code-test c to (safe) rust translation using llms and dynamic analysis," *arXiv preprint arXiv:2412.14234*, 2024.
- [39] H. Zhou, Y. Luo, M. Zhang, and D. Xu, "C2rusttv: An llm-based framework for c to rust translation and validation," in *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 2025, pp. 1254–1259.
- [40] Y. Bai and T. Palit, "Rustassure: Differential symbolic testing for llm-transpiled c-to-rust code," in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2025, pp. 534–546.
- [41] H. Sim, H. Cho, A. Shokri, Z. Fu, and B. Ravindran, "Encrust: Encapsulated substitution and agentic refinement on a live scaffold for safe c-to-rust translation," *arXiv preprint arXiv:2604.04527*, 2026.
- [42] M. Farrukh, B. Coskun, T. Palit, and M. Polychronakis, "Orbit: Guided agentic orchestration for autonomous c-to-rust transpilation," 2026. [Online]. Available: <https://arxiv.org/abs/2604.12048>
- [43] Y. Yan, Y. Feng, J. Liu, D. Liu, Z. Liu, H. Teng, and B. Xu, "C2rustxw: Program-structure-aware c-to-rust translation via program analysis and llm," *arXiv preprint arXiv:2603.28686*, 2026.
- [44] C. Hu, W. Zeng, Y. Shi, B. Shen, and X. Gu, "In line with context: Repository-level code generation via context inlining," *Proc. ACM Softw. Eng.*, vol. 3, no. FSE, pp. 1469–1491, 2026. [Online]. Available: <https://doi.org/10.1145/3797094>
- [45] Y. Gao, C. Wang, P. Huang, X. Liu, M. Zheng, and X. Zhang, "Raw pointer rewriting with llms for translating c to safer rust," 2026. [Online]. Available: <https://arxiv.org/abs/2505.04852>
- [46] T. Zhou, Z. Zhang, H. Lin, S. Jha, M. Christodorescu, K. Levchenko, and V. Chandrasekaran, "Sactor: Llm-driven correct and idiomatic c to rust translation with static analysis and ffi-based verification," *arXiv preprint arXiv:2503.12511*, 2025.
- [47] R. Li, B. Wang, T. Li, P. Saxena, and A. Kundu, "Translating c to rust: Lessons from a user study," *arXiv preprint arXiv:2411.14174*, 2024.
- [48] A. Valenzuela, M. Gonzalez-Mallo, C. Gutierrez, D. Garcia-Gasulla, G. Kestor, and S. Royuela, "From c to rust: Evaluating llm capabilities in transpilation through compilation errors," in *International Conference on High Performance Computing*. Springer, 2025, pp. 311–324.
- [49] B. Tadesse, V. Nitin, M. Salah, B. Ray, M. d'Amorim, and W. Assunção, "Code quality analysis of translations from c to rust," *arXiv preprint arXiv:2602.00840*, 2026.
- [50] N. Rutherford and D. O'Keeffe, "An empirical study of c to rust translation using local large-language models," in *The Third International Workshop on Large Language Models for Code*, 2026.
- [51] Y. Fu, D.-K. Kim, J. Kim, S. Sohn, L. Logeswaran, K. Bae, and H. Lee, "Autoguide: Automated generation and selection of context-aware guidelines for large language model agents," *Advances in Neural Information Processing Systems*, vol. 37, pp. 119919–119948, 2024.
- [52] E. Feng, W. Zhou, Z. Liu, L. Chen, Y. Dong, C. Zhang, Y. Zhao, D. Du, Z. Hua, Y. Xia *et al.*, "Get experience from practice: Llm agents with record & replay," *arXiv preprint arXiv:2505.17716*, 2025.
- [53] X. Tang, T. Qin, T. Peng, Z. Zhou, D. Shao, T. Du, X. Wei, P. Xia, F. Wu, H. Zhu *et al.*, "Agent kb: Leveraging cross-domain experience for agentic problem solving," *arXiv preprint arXiv:2507.06229*, 2025.
- [54] S. Gao, W. Zeng, Z. Yu, J. Wangni, C. Wang, K. Cai, S. He, and M. R. Lyu, "Swe-mem: Learning adaptive memory management for long-horizon coding agents," *CoRR*, vol. abs/2606.28434, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2606.28434>
- [55] W. Zeng, Y. Shi, X. Gu, C. Hu, C. Wang, Y. Cui, H. Zhou, M. Qi, J. Wangni, Z. Yu, S. Gao, K. Cai, and S. He, "Dockerless: Environment-free program verifier for coding agents," *CoRR*, vol. abs/2606.28436, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2606.28436>